

**ENUNCIADO DEL PROBLEMA DE PROYECTO
INTEGRADOR
Desarrollo de Sistemas Web**

Desarrollo de una Aplicación Web Dinámica con Arquitectura MVC

1. Planteamiento del Problema

En la actualidad, las organizaciones dependen de aplicaciones web para automatizar procesos, ofrecer servicios en línea y gestionar información de manera eficiente. Sitios como sistemas de soporte técnico, plataformas de comercio electrónico, portales académicos y sistemas de reservaciones integran múltiples tecnologías para proporcionar experiencias dinámicas e interactivas a los usuarios.

Detrás de estas aplicaciones existe una arquitectura que separa claramente la interfaz de usuario, la lógica del negocio y el acceso a los datos. Esta separación, implementada comúnmente mediante el patrón de diseño Modelo–Vista–Controlador (MVC), permite desarrollar sistemas más mantenibles, escalables y reutilizables.

En este contexto, se plantea el desarrollo de una aplicación web dinámica completa que integre:

- Estructura semántica con HTML5.
- Presentación visual responsiva con CSS3.
- Interactividad del lado del cliente con JavaScript.
- Comunicación asíncrona con AJAX y fetch().
- Persistencia de datos en MySQL.
- Backend con Node.js y Express.
- Motor de vistas EJS.
- Arquitectura MVC.

El proyecto permitirá a los estudiantes aplicar de forma integradora los conocimientos adquiridos durante el curso, desarrollando un sistema funcional similar a los utilizados en entornos profesionales.

2. Objetivo General

Diseñar, desarrollar e implementar una aplicación web dinámica que resuelva una problemática específica, empleando tecnologías frontend y backend modernas, base de datos relacional y arquitectura MVC.

3. Descripción General del Proyecto

Cada equipo desarrollará un sistema web sobre una temática elegida, que incluirá funcionalidades de registro y consulta de información.

**ENUNCIADO DEL PROBLEMA DE PROYECTO
INTEGRADOR
Desarrollo de Sistemas Web**

La aplicación deberá contar con:

1. Interfaz web responsiva.
2. Formularios de captura de información.
3. Validación en cliente y servidor.
4. Carga dinámica de datos mediante AJAX.
5. Persistencia en MySQL.
6. Arquitectura MVC.
7. Subida de archivos.
8. Dashboard o sección de indicadores.
9. Seguridad

4. Temáticas Sugeridas

Cada equipo elegirá una temática. Los temas posibles son:

1. Sistema de reporte de incidentes de ciberseguridad.
2. Sistema de reservación de equipos.
3. Plataforma de cursos y talleres.
4. Catálogo de servicios tecnológicos.
5. Tienda en línea.
6. Sistema de soporte técnico.
7. Sistema de inventario.

5. Requisitos Funcionales Obligatorios

5.1 Página Principal

La aplicación deberá incluir una página de inicio con:

- Encabezado con logotipo y menú de navegación.
- Banner o sección principal.
- Secciones informativas.
- Pie de página.

5.2 Gestión de Registros (CR)

El sistema deberá permitir:

- Registrar nuevos elementos.

**ENUNCIADO DEL PROBLEMA DE PROYECTO
INTEGRADOR
Desarrollo de Sistemas Web**

- Consultar registros.

5.3 Base de Datos

La base de datos deberá contener:

- Mínimo 3 tablas relacionadas.
- Integridad referencial mediante llaves primarias y foráneas.
- Datos de prueba.

5.4 Formularios

Los formularios deberán incluir:

- Controles HTML apropiados.
- Validación con JavaScript.
- Validación en servidor.
- Mensajes de error claros.

5.5 Subida de Archivos

La aplicación deberá permitir adjuntar al menos un archivo.

5.6 AJAX y JSON

Al menos una funcionalidad deberá utilizar `fetch()` para cargar o enviar datos sin recargar la página.

5.7 Dashboard o Indicadores

Mostrar estadísticas dinámicas utilizando información de la base de datos.

5.8 Manejo de Errores

Implementar control de errores tanto en cliente como en servidor.

**ENUNCIADO DEL PROBLEMA DE PROYECTO
INTEGRADOR
Desarrollo de Sistemas Web**

6. Requisitos Técnicos Obligatorios

Frontend

- HTML5 semántico
- CSS3
- Diseño responsivo
- JavaScript
- AJAX (fetch)
- JSON
- Bootstrap (opcional pero recomendado)

Backend

- Node.js
- Express
- EJS

Base de Datos

- MySQL

Arquitectura

- MVC

7. Requisitos de Seguridad en el Desarrollo de la Aplicación

Con el propósito de incorporar los principios fundamentales de seguridad en el desarrollo de aplicaciones web, cada equipo deberá implementar y documentar al menos **cuatro mecanismos de seguridad** en su proyecto final.

Estos mecanismos deberán ser funcionales, demostrables durante la presentación del proyecto y explicados técnicamente en el reporte final.

7.1 Validación de Entradas

Todos los formularios del sistema deberán validar la información ingresada por el usuario tanto en el cliente (JavaScript) como en el servidor (Node.js/Express).

**ENUNCIADO DEL PROBLEMA DE PROYECTO
INTEGRADOR
Desarrollo de Sistemas Web**

Se deberán validar, al menos:

- Campos obligatorios.
- Formato de correo electrónico.
- Longitud mínima y máxima.
- Tipos de datos numéricos.
- Restricciones de archivos subidos.

Propósito de seguridad: Prevenir datos inválidos y reducir la superficie de ataque.

7.2 Prevención de SQL Injection

Todas las consultas a MySQL deberán realizarse mediante consultas parametrizadas utilizando el paquete mysql2.

No se permitirá construir consultas concatenando directamente cadenas proporcionadas por el usuario.

Ejemplo correcto:

```
const [rows] = await pool.query(  
  "SELECT * FROM usuario WHERE correo = ?",  
  [correo]  
);
```

Propósito de seguridad: Evitar manipulación maliciosa de consultas SQL.

7.3 Prevención de Cross-Site Scripting (XSS)

La aplicación deberá evitar la ejecución de código JavaScript inyectado por usuarios.

Para ello:

- Los datos mostrados en las vistas EJS deberán utilizar `<%= %>` (escape automático).
- No se deberá utilizar `<%- %>` con contenido proveniente del usuario.
- Se deberá validar y sanitizar el contenido cuando sea necesario.

**ENUNCIADO DEL PROBLEMA DE PROYECTO
INTEGRADOR
Desarrollo de Sistemas Web**

Propósito de seguridad: Impedir la ejecución de scripts maliciosos en el navegador.

7.4 Manejo Seguro de Configuración

Las credenciales y parámetros sensibles deberán almacenarse en un archivo .env, por ejemplo:

```
DB_HOST=localhost  
DB_DATABASE=proyecto  
DB_USER=proyecto_user  
DB_PASSWORD=*****  
SERVER_PORT=3000
```

El archivo .env no deberá contenerse en repositorios públicos.

Propósito de seguridad: Proteger información sensible y facilitar el despliegue.

7.5 Control de Archivos Subidos

La aplicación deberá implementar al menos las siguientes validaciones:

- Restricción por extensión.
- Restricción por tipo MIME.
- Límite de tamaño.
- Generación automática de nombres seguros.

Propósito de seguridad: Evitar la carga de archivos maliciosos.

7.6 Manejo de Errores

La aplicación deberá mostrar mensajes amigables al usuario y evitar revelar información sensible como:

- Consultas SQL,
- Rutas del sistema,
- Credenciales,
- Stack traces.

En modo desarrollo se podrán mostrar detalles adicionales.

Propósito de seguridad: Reducir fuga de información.

**ENUNCIADO DEL PROBLEMA DE PROYECTO
INTEGRADOR
Desarrollo de Sistemas Web**

La sección deberá contener la **explicación de mecanismos de seguridad**:

1. Amenazas identificadas.
2. Controles implementados.
3. Fragmentos de código relevantes.
4. Evidencia de pruebas.
5. Reflexión sobre riesgos mitigados.

8. Metodología de Desarrollo del Proyecto

El desarrollo del proyecto final deberá seguir un proceso metodológico estructurado, con el propósito de que el equipo documente de manera sistemática las decisiones de análisis, diseño, implementación y evaluación del sistema.

La metodología de desarrollo formará parte de la evaluación del proyecto final. No se calificará únicamente el resultado funcional, sino también la capacidad del equipo para justificar y documentar el proceso seguido desde la concepción del sistema hasta su implementación y validación.

La metodología para emplear estará basada en tres enfoques complementarios:

- **Desarrollo Ágil (Agile):** Para organizar el trabajo de manera incremental e iterativa.
- **Diseño Centrado en el Usuario (User-Centered Design, UCD):** Para asegurar que la aplicación responda a las necesidades de los usuarios.
- **Arquitectura de la Información (Information Architecture):** Para estructurar adecuadamente el contenido y la navegación del sistema.

Fase 1. Definición del Proyecto

Objetivo: Definir el problema que resolverá la aplicación y caracterizar a los usuarios principales.

Actividades mínimas:

1. Definir el nombre del proyecto.
2. Describir la problemática que se pretende resolver.
3. Establecer el objetivo general del sistema.
4. Identificar el público objetivo.
5. Elaborar al menos un perfil de usuario (User Persona).
6. Identificar los requerimientos funcionales iniciales.

**ENUNCIADO DEL PROBLEMA DE PROYECTO
INTEGRADOR
Desarrollo de Sistemas Web**

Artefactos esperados:

- Descripción del proyecto.
- Objetivo general.
- User Persona.
- Lista preliminar de requisitos.

Fase 2. Arquitectura de la Información

Objetivo: Organizar el contenido y definir la estructura de navegación del sistema.

Actividades mínimas:

1. Construir el mapa del sitio.
2. Diseñar el menú de navegación.

Artefactos esperados:

- Mapa del sitio.
- Estructura de navegación.

Fase 3. Diseño de la Experiencia del Usuario

Objetivo: Diseñar la estructura visual y funcional de las páginas principales.

Actividades mínimas:

1. Elaborar wireframes de:
 - Página principal.
 - Formulario principal.
 - Vista de listado o dashboard.
2. Definir paleta de colores y tipografía.

Artefactos esperados:

- Wireframes.

Fase 4. Diseño de la Base de Datos

Objetivo: Modelar la estructura relacional necesaria para soportar la aplicación.

**ENUNCIADO DEL PROBLEMA DE PROYECTO
INTEGRADOR
Desarrollo de Sistemas Web**

Actividades mínimas:

1. Elaborar el Modelo Relacional (Esquemas de relaciones, Diagrama de la base de datos)
2. Crear script SQL.

Artefactos esperados:

- Modelo relacional
- Script bd.sql.

Fase 5. Implementación del Frontend

Objetivo: Construir la interfaz de usuario responsiva e interactiva.

Actividades mínimas:

1. Implementar HTML semántico.
2. Aplicar CSS y Bootstrap (no obligatorio pero recomendado).
3. Incorporar JavaScript para DOM y eventos.
4. Validar formularios.
5. Implementar AJAX con fetch().

Artefactos esperados:

- Archivos HTML/EJS.
- Hojas de estilo. (si aplica)
- Scripts JavaScript.

Fase 6. Implementación del Backend

Objetivo: Desarrollar la lógica del servidor siguiendo arquitectura MVC.

Actividades mínimas:

1. Configurar Node.js y Express.
2. Implementar modelos.
3. Implementar controladores.
4. Configurar rutas.
5. Integrar MySQL.
6. Implementar operaciones Create/Read.

**ENUNCIADO DEL PROBLEMA DE PROYECTO
INTEGRADOR
Desarrollo de Sistemas Web**

7. Configurar manejo de errores.

Artefactos esperados:

- Proyecto MVC funcional.
- Archivo .env.
- package.json.

Fase 7. Pruebas y Evaluación

Objetivo: Verificar el correcto funcionamiento del sistema.

Actividades mínimas:

1. Probar todos los módulos.
2. Verificar validaciones.
3. Revisar diseño responsivo.
4. Evaluar manejo de errores.
5. Corregir defectos.

Artefactos esperados:

- Evidencias de pruebas
 - Versión final estable

Fase 8. Documentación y Presentación

Objetivo: Integrar la documentación técnica y preparar la exposición final.

Actividades mínimas:

1. Elaborar reporte técnico
2. Organizar código fuente
3. Preparar presentación
4. Ensayar demostración

Artefactos esperados:

**ENUNCIADO DEL PROBLEMA DE PROYECTO
INTEGRADOR
Desarrollo de Sistemas Web**

- Documento PDF final
- Código fuente
- Presentación

9. RESULTADOS ENTREGABLES

Cada equipo deberá entregar, a través de EMINUS, un archivo comprimido en formato .zip con el nombre:

DSW-ProyectoFinal-EquipoXX.zip

donde XX corresponde al nombre de equipo.

El archivo comprimido deberá contener, de manera organizada, todos los elementos descritos a continuación.

9.1 Código Fuente Completo del Proyecto

Carpeta con el código fuente funcional de la aplicación, incluyendo como mínimo:

```
ProyectoFinal/  
├── controllers/  
├── data/  
├── middlewares/  
├── models/  
├── routes/  
├── views/  
├── public/  
│   ├── css/  
│   ├── js/  
│   ├── img/  
│   └── uploads/  
├── .env.example  
├── bd.sql  
├── package.json  
├── package-lock.json  
├── README.md  
└── index.js
```

Nota: El archivo .env con credenciales reales no debe incluirse. En su lugar, deberá entregarse un archivo .env.example con valores de referencia.

**ENUNCIADO DEL PROBLEMA DE PROYECTO
INTEGRADOR
Desarrollo de Sistemas Web**

9.2 Script de Base de Datos

Archivo:

bd.sql

El script deberá incluir:

- Creación de la base de datos.
- Creación de tablas.
- Llaves primarias y foráneas.
- Datos de prueba suficientes para demostrar la funcionalidad.

9.3 Documento de Planeación y Diseño del Proyecto

Archivo en formato PDF con el nombre:

Planeacion_Proyecto_Final.pdf

Deberá incluir, al menos:

1. Portada
2. Información general del proyecto.
3. Planteamiento del problema.
4. Objetivo general.
5. User Persona.
6. Requisitos funcionales.
7. Arquitectura de la información.
8. Mapa del sitio.
9. Wireframes.
10. Paleta de colores y tipografía.
11. Modelo relacional.
12. Diagrama del stack tecnológico.
13. Estructura MVC del proyecto.
14. Modelado básico de amenazas.
15. Evidencias de pruebas.
16. Reflexión final.

9.4 Reporte Técnico Final

Archivo en formato PDF con el nombre:

**ENUNCIADO DEL PROBLEMA DE PROYECTO
INTEGRADOR
Desarrollo de Sistemas Web**

Reporte_Tecnico_Final.pdf

Deberá incluir:

1. Portada.
2. Introducción.
3. Descripción de la solución.
4. Tecnologías empleadas.
5. Explicación de la arquitectura MVC.
6. Descripción de módulos implementados.
7. Explicación de mecanismos de seguridad.
8. Problemas encontrados y soluciones.
9. Conclusiones.
10. Referencias en formato APA.

Si se desea, el documento de planeación y el reporte técnico podrán integrarse en un solo PDF.

9.5 Presentación Ejecutiva

Archivo en formato PowerPoint o PDF:

Presentacion_Proyecto_Final.pdf

Duración estimada de exposición: 10 minutos.

La presentación deberá incluir:

1. Problema y objetivos.
2. Arquitectura MVC del sistema.
3. Funcionalidades principales.
4. Mecanismos de seguridad.
5. Demostración del sistema.
6. Conclusiones.

9.6 Archivo README

Archivo:

README.md

**ENUNCIADO DEL PROBLEMA DE PROYECTO
INTEGRADOR
Desarrollo de Sistemas Web**

Con instrucciones claras para:

1. Instalar dependencias.
2. Configurar .env.
3. Crear la base de datos.
4. Ejecutar la aplicación.
5. Acceder al sistema.

Ejemplo de comandos:

```
npm install  
npm run dev
```

9.7 Evidencias de Validación

Capturas de pantalla que demuestren:

- Diseño responsivo.
- Validaciones de formularios.
- Uso de AJAX.
- Operaciones Create/Read.
- Subida de archivos.
- Dashboard o indicadores.
- Mecanismos de seguridad.

9.8 Lista de Verificación Final

Antes de entregar, el equipo deberá verificar que:

- La aplicación ejecuta correctamente.
- La página de inicio contiene los elementos solicitados
- Se emplea la tecnología solicitada
- Se siguen las fases de la metodología de desarrollo indicada
- La base de datos se crea con bd.sql.
- El archivo .env.example está incluido.
- El código está organizado bajo MVC.
- Se implementa Create/Read completo.
- Existe al menos una funcionalidad AJAX.
- Se aplican mecanismos de seguridad.
- El diseño es responsivo.
- Se adjunta la documentación solicitada.

**ENUNCIADO DEL PROBLEMA DE PROYECTO
INTEGRADOR
Desarrollo de Sistemas Web**

- El archivo .zip se puede descomprimir y ejecutar sin errores.